CS247 Assignment 4 Report

GPU Volume Raycasting: DVR and Iso-Surface in GLSL

1. Overview

Assignment 4 is a complete architecture change from Assignments 1 to 3. No more geometric extraction (marching squares or cubes), no more GLFW plus GLAD plus core-profile setup. Instead, every frame the program

1. Renders the front faces of a unit cube into an FBO whose RGB encodes the cube's tex coord at each pixel (ray entry point);
2. Renders the back faces the same way (ray exit point);
3. Draws a screen-filling quad whose fragment shader walks the ray from entry to exit, sampling the 3D volume texture and either accumulating colours via a transfer function (DVR) or stopping at the first iso crossing (Iso surface).

The host code was already nearly complete. The entire assignment lives in two small GLSL files and one C++ function:

Implemented in	File
2D windowing transfer function (20 pts)	`Source/main.cpp::UpdateTransferfunction`
Iso-surface raycasting + shading + ERT (20 pts)	`shaders/raycast_iso.glsl`
DVR + opacity correction + shading + ERT (35 pts)	`shaders/raycast_dvr.glsl`
MIP (bonus +5)	same DVR shader, gated on the `mip` uniform
Uniform wiring (textures + scalars)	`Source/main.cpp::RenderRaycastPass`

The reference screenshots from the upstream `README.md` were used as ground truth: TF0 with and without shading, TF5, iso value 0.2 and 0.3.

2. Requirements coverage

#	Requirement (from `README.md`)	Points	Where & how
1	DVR in GLSL fragment shader	20	`raycast_dvr.glsl`. Front-to-back compositing over `glDrawArrays`-equivalent fullscreen quad.
1a	Opacity correction	5	`src.a = 1 - pow(1 - src.a, step_size / REF_STEP)` (Beer-Lambert correction for the actual step size).
1b	Shading from central-difference gradient	5	`sampleGradient()` plus Lambertian `max(N·L,0) + ambient`, with optional gradient-magnitude modulation (`g` key).
1c	Early ray termination	5	`if (accum.a > 0.95) break;`
2	Interactive windowing transfer function	20	`UpdateTransferfunction()` rebuilds the 128-entry `tf0[]` from `tf_win_min`, `tf_win_max` and uploads it.
Total		**100**	

Bonus implemented: MIP (+5). Implemented inside `raycast_dvr.glsl` and gated by the existing `l` key toggle (`mip` uniform).

Not implemented: pre-integrated TF, empty-space skipping, axis-aligned clipping planes.

3. The two-pass FBO setup (host side, already in template)

The cube `Draw()` function is called twice per frame:

```
RenderFrontFaces(x, y, z);   // glCullFace(GL_BACK) → only front faces survive
RenderBackFaces (x, y, z);   // glCullFace(GL_FRONT) → only back faces survive
```

Each cube vertex is written with `glColor3f(texCoord.x, texCoord.y, texCoord.z)` before `glVertex3f(position)`. With the fixed-function pipeline, this makes the interpolated vertex colour at every pixel equal to the texture coordinate at that pixel. Two RGBA16F FBOs (one each for front and back) end up containing the 3D point where every screen pixel's ray enters and exits the volume.

In the raycast pass these two FBOs are bound as `sampler2D frontfaces` and `backfaces` (texture units 1 and 2) and the volume is bound as `sampler3D volume` (unit 3). The transfer function stays on unit 0.

4. The iso-surface shader walk-through

`shaders/raycast_iso.glsl`:

```
vec2 uv       = gl_TexCoord[0].xy;
vec3 rayStart = texture2D(frontfaces, uv).xyz;
vec3 rayEnd   = texture2D(backfaces,  uv).xyz;

vec3  dir     = normalize(rayEnd - rayStart);
vec3  step    = dir * step_size;
int   steps   = int(length(rayEnd - rayStart) / step_size) + 2;

float prev = texture3D(volume, rayStart).r;
vec3  pos  = rayStart;
for (int i = 1; i < 2048; ++i) {
    if (i >= steps) break;
    pos += step;
    float v = texture3D(volume, pos).r;

    if ((prev < iso) != (v < iso)) {                      // sign change?
        float t       = (iso - prev) / (v - prev + 1e-7);  // linear refine
        vec3  hit     = pos - step + t * step;
        vec3  N       = -normalize(sampleGradient(hit));   // outward
        vec3  L       = normalize(vec3(1.0));
        vec3  color   = vec3(0.85, 0.9, 1.0) *
                        clamp(ambient + max(dot(N,L),0.0), 0.0, 1.0);
        gl_FragColor  = vec4(color, 1.0);
        return;                                           // EARLY-RT
    }
    prev = v;
```

```
}
discard;
```

Pixels outside the cube projection (where `rayStart == rayEnd`) are discarded via the `length(ray) < 1e-5` guard at the top.

5. The DVR shader walk-through

`shaders/raycast_dvr.glsl`:

```
vec4 accum = vec4(0.0);
vec3 pos   = rayStart;
const float REF_STEP = 1.0 / 200.0;

for (int i = 0; i < 2048; ++i) {
    if (i >= steps) break;
    float v   = texture3D(volume, pos).r;
    vec4  src = texture1D(transferfunction, v);

    // 5.1 OPACITY CORRECTION
    src.a = 1.0 - pow(max(1.0 - src.a, 0.0), step_size / REF_STEP);

    if (src.a > 0.001) {
        // 5.2 SHADING (per sample)
        if (enable_lighting == 1) {
            vec3  grad = sampleGradient(pos);
            vec3  N    = -normalize(grad + vec3(1e-7));
            float diff = max(dot(N, L), 0.0);
            float shade = clamp(ambient + diff, 0.0, 1.0);
            if (enable_gm_scaling == 1)
                shade *= clamp(length(grad) * 4.0, 0.0, 1.0);
            src.rgb *= shade;
        }
        // 5.3 FRONT-TO-BACK COMPOSITING
        accum.rgb += (1.0 - accum.a) * src.a * src.rgb;
        accum.a   += (1.0 - accum.a) * src.a;
        // 5.4 EARLY RAY TERMINATION
        if (accum.a > 0.95) break;
    }
    pos += step;
}
gl_FragColor = accum;
```

Opacity correction uses the Beer-Lambert relation `α_corrected = 1 − (1 − α_ref)^(Δs / Δs_ref)`. The reference step is `1/200`, the same scale as the `+` and `−` keys would land at the third press. This keeps the rendered density invariant when the user changes `step_size` with `+` or `−`.

Gradient magnitude modulation (`g` key) multiplies the shading by `clamp(|∇F| · 4, 0, 1)`, which suppresses interior homogenous regions and highlights edges. Useful for emphasising the skin and bone interface in TF0.

MIP (bonus)

When `mip == 1` the shader replaces the compositing loop with a maximum value pass and looks up the max once at the end through the current TF:

```
float maxV = 0.0;
for (i = 0; i < steps; ++i) { maxV = max(maxV, texture3D(volume, p).r); p +=
step; }
gl_FragColor = vec4(texture1D(transferfunction, maxV).rgb, 1.0);
```

6. Windowing transfer function (host side)

`Source/main.cpp::UpdateTransferfunction():`

```
for (int i = 0; i < 128; ++i) {
    float t = 0.0f;
    if (i >= tf_win_min && i <= tf_win_max)
        t = (i - tf_win_min) / max(tf_win_max - tf_win_min, 1);
    tf0[4i+0..3] = vec4(t, t, t, t);        // grey ramp + linear opacity
}
glTexImage1D(GL_TEXTURE_1D, 0, GL_RGBA, 128, 0, GL_RGBA, GL_FLOAT, tf0);
```

Inside the window the TF is a linear ramp from black or transparent to white or opaque. Outside, the TF is fully transparent. Squeezing the window narrows the visible intensity range. The rebuild plus re-upload is cheap (128 RGBA floats) so the response is instant.

The `w` and `s` keys move `tf_win_min`. The `e` and `d` keys move `tf_win_max`. Their handlers already call `UpdateTransferfunction()`, which uploads to `tf_texture`, exactly the texture the DVR shader is sampling.

7. Host-side wiring (`RenderRaycastPass`)

The original template only bound the transfer function and a debug `params` vec4. The actual algorithms need four textures and a handful of scalars:

Unit	Sampler	Bound to
0	`sampler1D transferfunction`	`tf_texture`
1	`sampler2D frontfaces`	`frontface_buffer`
2	`sampler2D backfaces`	`backface_buffer`
3	`sampler3D volume`	`vol_texture`

Plus the scalars `step_size`, `iso_value`, `ambient`, the toggle integers `enable_lighting`, `enable_gm_scaling`, `mip`, and the 3D `vol_dim` (needed for the 1-voxel offsets used by the central difference gradient).

8. Build & run

```
brew install glew                       # one-time
cd Assignment_4
cmake -S . -B build
```

```
cmake --build build -j
./build/assignment
```

The window starts blank. Press **1** for `lobster.dat` (120 × 120 × 34) or **2** for `skewed_head.dat` (184 × 256 × 170).

9. Controls

Mode

Key	Action
`f`	Toggle DVR vs Iso surface
`l`	Toggle MIP (bonus mode, overrides DVR while on)
`5` to `0`	Pick TF0 to TF5 (TF0 is the windowing TF)

Iso surface tweaks

Key	Action
`i` / `k`	iso value `+0.05` / `−0.05`
`o`	Toggle Lambertian shading
`a` / `z`	Ambient `+0.05` / `−0.05`

DVR tweaks

Key	Action
`o`	Toggle per-sample shading
`g`	Toggle gradient-magnitude edge boost
`+` / `−`	Decrease / increase step size (smaller = sharper but slower)
`w` / `s`	TF0 lower window `+1` / `−1`
`e` / `d`	TF0 upper window `+1` / `−1`

View

Key	Action
Left-drag	Rotate
Ctrl + Left-drag	Z-axis rotate plus zoom
Arrows	Rotate
`Home`	Reset view
Space	Pause or resume auto rotation
`b`	Cycle background colour
`q`, `Esc`	Quit

10. Visual results

A screen recording of the implementation is included as `demo.mov`. It walks through: